P0019r5 : Atomic View

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0019r5
Date:	2017-03-06
Reply-to:	hcedwar@sandia.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Hans Boehm
Contact:	hboehm@google.com
Author:	Olivier Giroux
Contact:	ogiroux@nvidia.com
Author:	James Reus
Contact:	reus1@llnl.gov
Audience:	Library Evolution
URL:	https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0019.rst

Revision History

P0019r3

- Align proposal with content of corresponding sections in N5131, 2016-07-15.
- Remove the *one root wrapping constructor* requirement from **atomic_array_view**.
- Other minor revisions responding to feedback from SG1 @ Oulu.

P0019r4

- wrapper constructor strengthen requires clause and omit throws clause
- Note types must be trivially copyable, as required for all atomics
- 2016-11-09 Issaquah SG1 decision: move to LEWG targeting Concurrency TS V2

D0019r5

- 2017-03-01 Kona LEWG review
  - Merge in P0440 Floating Point Atomic View because LEWG consensus to move P0020 Floating Point Atomic to C++20 IS
  - Rename from **atomic_view** and **atomic_array_view** to **atomic_ref<T>** and **atomic_ref<T[]>**, other name suggested **atomic_wrapper**.
  - Remove **constexpr** qualification from default constructor because this qualification constrains implementations and does not add apparent value.
- Remove default constructor, copy constructor, and assignment operator for tighter alignment with **atomic<T>** and prevent empty references.
- Revise syntax to align with P0558r1, Resolving atomic<T> base class inconsistencies
- Recommend feature next macro

Overview / Motivation / Discussion

This paper proposes an extension to the atomic operations library [atomics] for atomic operations applied to non-atomic objects. As required by [atomics.types.generic] 20.5p1 the wrapped type **T** must be trivially copiable. This paper includes *atomic floating point* capability defined in P0020r5.

Motivation: Atomic Operations on a Single Non-atomic Object

An *atomic reference* is used to perform atomic operations on referenced non-atomic object. The intent is for *atomic reference* to provide the best-performing implementation of atomic operations for the non-atomic object type. All atomic operations

performed through an *atomic reference* on a referenced non-atomic object are atomic with respect to any other *atomic reference* that references the same object, as defined by equality of pointers to that object. The intent is for atomic operations to directly update the referenced object. The *atomic reference wrapping constructor* may acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. Such *atomic reference* objects are not lock-free and not address-free. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource.

Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with a **atomic** object.

An object may be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively **atomic** would incur an unnecessary performance penalty.

Motivation: Atomic Operations on Members of a Very Large Array

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An *atomic array reference* is used to perform atomic operations on the non-atomic members of the referenced array. The intent is for *atomic array reference* to provide the best-performing implementation of atomic operations for the members of the array.

Naming

The current **Header <atomic> synopsis** contains the following.

```
namespace std {
    template< class T > struct atomic;
    template< class T > struct atomic<T*>;
}
```

The proposal introduces the following type and partial specializations.

```
namespace std {
namespace experimental {

    template< class T > struct atomic_ref;
    template< class T > struct atomic_ref<T*>;
    template< class T > struct atomic_ref<T[]>;;

}}
```

Other possible naming or partial specializations

- atomic_wrapper< T >, atomic_wrapper< T * >, atomic_wrapper< T[] >
- atomic< T & >, atomic< T * & >, atomic< T[] >

Wrappability Constraints

The *wrapping constructor* of an atomic reference requires that the object be *wrappable* for atomic operations; for example that the object is properly aligned in memory or resides in GPU register memory. This revision of P0019 does not enumerate all potential *wrappability* constraints or specify behavior of the wrapping constructor when these constraints are violated. It is quality-of-implementation to generate appropriate error information.

Proposal

Recommended feature text macro

```
__cpp_lib_atomic_ref
```

add to Header <atomic> synopsis

```
namespace std {
namespace experimental {

    template< class T > struct atomic_ref ;
    template< class T > struct atomic_ref< T * >;
    template< class T > struct atomic_ref< T[] >;

}}
```

add section to Class template atomic

```
template< class T > struct atomic_ref {
    using value_type = T;
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( T , memory_order = memory_order_seq_cst ) const noexcept;
    T load( memory_order = memory_order_seq_cst ) const noexcept;
    operator T() const noexcept ;
    T exchange( T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( T&, T, memory_order = memory_order_seq_cst ) const noexcept;

    atomic_ref() = delete ;
    atomic_ref( const atomic_ref & ) = delete ;
    atomic_ref & operator = ( const atomic_ref & ) = delete ;

    explicit atomic_ref( T & obj ); // wrapping constructor

    T operator=(T) const noexcept ;
};
```

static constexpr size_t required_alignment = *implementation-defined* **;**

The required alignment of an object to be referenced by an atomic reference, which is at least `alignof(T)`. [Note: An architecture may support lock-free atomic operations on objects of type *T* only if those objects meet a required alignment. The intent is for *atomic_ref* to provide lock-free atomic operations whenever possible. For example, an architecture may be able to support lock-free operations on `std::complex<double>` only if aligned to `2*alignof(double)` and not `alignof(double)` . - end note]

atomic_ref(T & obj);

This wrapping constructor constructs an atomic reference that references the non-atomic object. Atomic operations applied to object through a referencing *atomic reference* are atomic with respect to atomic operations applied through any other *atomic reference* that references that *object*.

Requires: The referenced non-atomic object shall be aligned to `required_alignment`. The lifetime (3.8) of `*this` shall not exceed the lifetime of the referenced non-atomic object. While any `atomic_ref` instance exists that references the object all accesses of that object shall exclusively occur through those `atomic_ref` instances. If the referenced *object* is of a class or aggregate type then members of that object shall not be concurrently wrapped by an `atomic_ref` object. The referenced object shall not be a member of an array that is wrapped by an `atomic_ref<T[]>` . [Note: Other implementation dependent conditions may exist. - end note]

Effects: `*this` references the non-atomic object* [Note: The wrapping constructor may acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations applied to the referenced non-atomic object. - end note]

add to Specializations for integers

```
template<> struct atomic_ref< integral > {
    using value_type = integral ;
    using difference_type = value_type;
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( integral , memory_order = memory_order_seq_cst ) const noexcept;
    integral load( memory_order = memory_order_seq_cst ) const noexcept;
    operator integral () const noexcept ;
    integral exchange( integral , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( integral & , integral , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( integral & , integral , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( integral & , integral , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( integral &, integral , memory_order = memory_order_seq_cst ) const noexcept;

    integral fetch_add( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_sub( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_and( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_or( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_xor( integral , memory_order = memory_order_seq_cst) const noexcept;

    atomic_ref() = delete ;
    atomic_ref( const atomic_ref & ) = delete ;
    atomic_ref & operator = ( const atomic_ref & ) = delete ;

    explicit atomic_ref( integral & obj ); // wrapping constructor

    integral operator=( integral ) const noexcept ;
    integral operator++(int) const noexcept;
    integral operator--(int) const noexcept;
    integral operator++() const noexcept;
    integral operator--() const noexcept;
    integral operator+=( integral ) const noexcept;
    integral operator-=( integral ) const noexcept;
    integral operator&=( integral ) const noexcept;
    integral operator|=( integral ) const noexcept;
    integral operator^=( integral ) const noexcept;
};
```

add to Specializations for floating-point

```
template<> struct atomic_ref< floating-point > {
    using value_type = floating-point ;
    using difference_type = value_type;
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( floating-point , memory_order = memory_order_seq_cst ) const noexcept;
    floating-point load( memory_order = memory_order_seq_cst ) const noexcept;
    operator floating-point () const noexcept ;
    floating-point exchange( floating-point , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order = memory_order_seq_cst ) const
    noexcept;
    bool compare_exchange_strong( floating-point &, floating-point , memory_order = memory_order_seq_cst ) const
    noexcept;

    floating-point fetch_add( floating-point , memory_order = memory_order_seq_cst) const noexcept;
    floating-point fetch_sub( floating-point , memory_order = memory_order_seq_cst) const noexcept;
```

```
    atomic_ref() = delete ;
    atomic_ref( const atomic_ref & ) = delete ;
    atomic_ref & operator = ( const atomic_ref & ) = delete ;

    explicit atomic_ref( floating-point & obj ) noexcept ;

    floating-point operator=( floating-point ) noexcept ;
    floating-point operator+=( floating-point ) const noexcept ;
    floating-point operator-=( floating-point ) const noexcept ;
};
```

add to Partial specializations for pointers

```
template<class T> struct atomic_ref< T * > {
    using value_type = T * ;
    using difference_type = ptrdiff_t;
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( T * , memory_order = memory_order_seq_cst ) const noexcept;
    T * load( memory_order = memory_order_seq_cst ) const noexcept;
    operator T * () const noexcept ;
    T * exchange( T * , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( T * & , T * , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( T * & , T * , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( T * & , T * , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( T * &, T * , memory_order = memory_order_seq_cst ) const noexcept;

    T * fetch_add( difference_type , memory_order = memory_order_seq_cst) const noexcept;
    T * fetch_sub( difference_type , memory_order = memory_order_seq_cst) const noexcept;

    ~atomic_ref();
    atomic_ref() = delete ;
    atomic_ref( const atomic_ref & ) = delete ;
    atomic_ref & operator = ( const atomic_ref & ) = delete ;

    explicit atomic_ref( T * & obj ); // wrapping constructor

    T * operator=( T * ) const noexcept ;
    T * operator++(int) const noexcept;
    T * operator--(int) const noexcept;
    T * operator++() const noexcept;
    T * operator--() const noexcept;
    T * operator+=( difference_type ) const noexcept;
    T * operator-=( difference_type ) const noexcept;
};
```

add to Partial specializations for elements of array of unknown bound

Performing compatibility verification within the atomic reference wrapping constructor for many elements in an array can introduce unecessary, redundant operations. The *atomic array reference* partial specialization wraps the entire array such that compatibility verification can be performed once and amortized among all atomic references to members of that array.

```
template< class T > struct atomic_ref< T[] > {

    static constexpr size_t required_alignment = implementation defined ;
    static constexpr bool is_always_lock_free = implementation defined ;
    bool is_lock_free() const noexcept ;

    atomic_ref() = delete ;
    atomic_ref( const atomic_ref & ) = delete ;
```

```
    atomic_ref & operator = ( const atomic_ref & ) = delete ;

    atomic_array_ref( T * , size_t ); // wrapping constructor

    size_t size() const noexcept ;

    atomic_ref<T> operator[]( size_t ) const noexcept;
};
```

atomic_ref(T * array , size_t length);

This wrapping constructor constructs an `atomic_ref<T[]>` that references an array of non-atomic elements spanning `[array..array+length)`.

Requires: The referenced non-atomic array shall be aligned to `required_alignment`. The lifetime (3.8) of `*this` shall not exceed the lifetime of the referenced non-atomic array. All `atomic_ref<T[]>` instances that reference any element of the array shall reference the same span of the array. As long as any `atomic_ref<T[]>` instance exists that references array all accesses to members of that array shall exclusively occur through those `atomic_ref<T[]>` instances. No element of array is concurrently wrap constructed by an `atomic_ref<T>`. [Note: Other implementation dependent conditions may exist. - end note]

Effects: `*this` references the non-atomic array. Atomic operations on members of array are atomic with respect to atomic operations on members referenced through any other `atomic_ref<T[]>` instance. [Note: The *wrapping constructor* may acquire shared resources, such as a locks associated with the referenced array, to enable atomic operations applied to the referenced non-atomic members of referenced array. - end note]

atomic_ref<T> operator[](size_t i) const noexcept ;

Requires: `i < size()` and the lifetime of the returned `atomic_ref<T>` shall not exceed the lifetime of the associated `atomic_ref<T[]>`. [Note: Analogous to the lifetime of an iterator with respect to the lifetime of the associated container. - end note]

Example usage:

```
// atomic reference wrapper constructor:
atomic_ref<T[]> array( ptr , N );

// atomic operation on a member:
array[i].atomic-operation(...);

// atomic operations through a temporary value
// within a concurrent function:
auto x = array[i];
x.atomic-operation-a(...);
x.atomic-operation-b(...);
```